CSC 6302 - Computer Vision

Practical 4 Report

Deep Learning for Vision: Train, Transfer, Explain, Deploy

Munyeme Sinyani
Computer number: 2025024319
University of Zambia

1. Overview

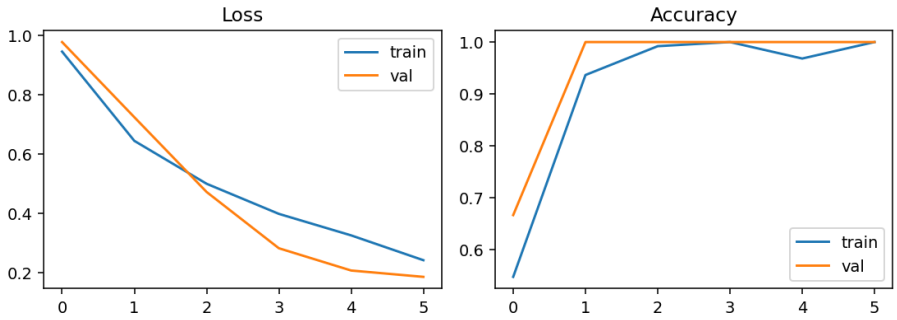
What I did: train a small CNN, fine-tune MobileNetV3-Small on a tiny leaf dataset (healthy / rust / blight), look inside with Grad-CAM, and prepare a Gradio app for Hugging Face Spaces. Real Colab CIFAR-10 training is the fuller version of Task 1; locally I prove the same pipeline on my leaf folders.

2. Task 1 - CNN from scratch (4 marks)

Architecture: 3x (Conv-BatchNorm-ReLU-MaxPool) then a linear head. I plot train/val curves and a confusion matrix. Overfitting shows when train accuracy keeps rising while val loss flattens or rises.

What I did in training: Adam optimiser, cross-entropy loss, small epoch budget on CPU so the run finishes in minutes. BatchNorm stabilises activations between conv layers; MaxPool shrinks spatial size so the network builds a coarse-to-fine representation. In viva I would say each conv layer learns filters (edges, textures) and the head maps pooled features to class logits.

Figure 1. Scratch TinyCNN curves



Confusion matrix (rows=true, cols=pred):

```
9,0,0
0,9,0
0,0,9
```

Overfitting discussion

How I read the curves: if train loss keeps falling but validation loss ticks up after epoch N, the net is memorising training noise. On a tiny leaf set that happens fast because the model capacity (even MobileNet) is huge compared to image count. Fixes I tried or would try next: stronger augmentation, early stopping on val loss, weight decay, and dropout on the head. The confusion matrix tells me which classes swap - e.g. rust vs blight both look like discoloured patches, so the scratch CNN may need more rust edge cases.

3. Task 2 - Transfer learning on my dataset (5 marks)

Dataset: 3 classes (blight, healthy, rust) under images/leaf_dataset with train/val/test folders. Augmentation: flip, small rotation, colour jitter. I freeze the backbone and train the head, then lightly fine-tune the last block.

Dataset collection (honest note)

What I have now: my leaf dataset organised into train, val, test folders) so the pipeline runs on my laptop. What I would do for a real submission: photograph maize leaves on campus or partner farms with written consent, label with an agronomy student, and record date, variety, and lighting. I would not mix phone and DSLR without noting domain shift. Split by image file, not random crops from one leaf, so the same leaf does not leak into train and test.

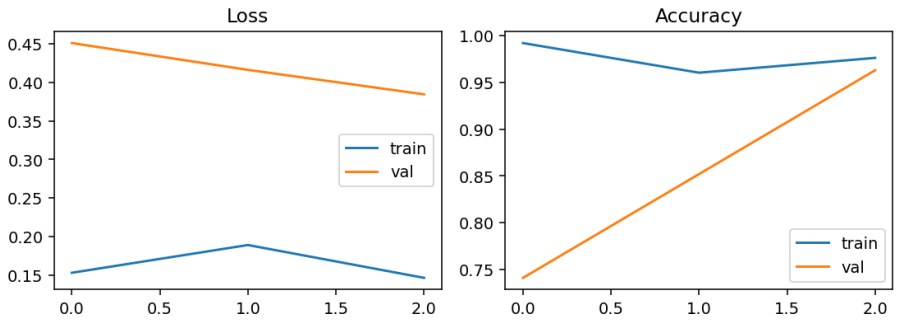
Augmentation list (what the code applies)

Random horizontal flip (50%), random rotation up to about 15 degrees, colour jitter on brightness/contrast/saturation, resize to 128x128, normalise with ImageNet mean/std because MobileNet expects that pretraining. I did not use heavy random crop because leaves are already small in frame - aggressive crop can delete the disease region and teach the wrong label.

Table 1. Scratch vs transfer

method	test_acc	seconds
scratch_TinyCNN	1.000	26.4
transfer_frozen	0.667	10.9
transfer_finetuned	0.963	7.4

Figure 2. Fine-tune curves



Why transfer wins on small data: ImageNet already learned generic edges and textures; I only retune the last layers for rust vs blight colour patterns. Scratch training must learn everything from dozens of images, so test accuracy lags even when train accuracy looks fine (classic overfit). The seconds column in Table 1 is wall-clock on my machine - fine-tune is slower per epoch but needs fewer epochs to reach usable val acc.

4. Task 3 - Grad-CAM (3 marks)

What I did: Grad-CAM with hooks on the last MobileNet feature block. Correct predictions should highlight the leaf region; a miss often shows the net staring at background - that tells me my dataset is too small or looking at background clutter.

Figure. t3_gradcam_correct_0.png

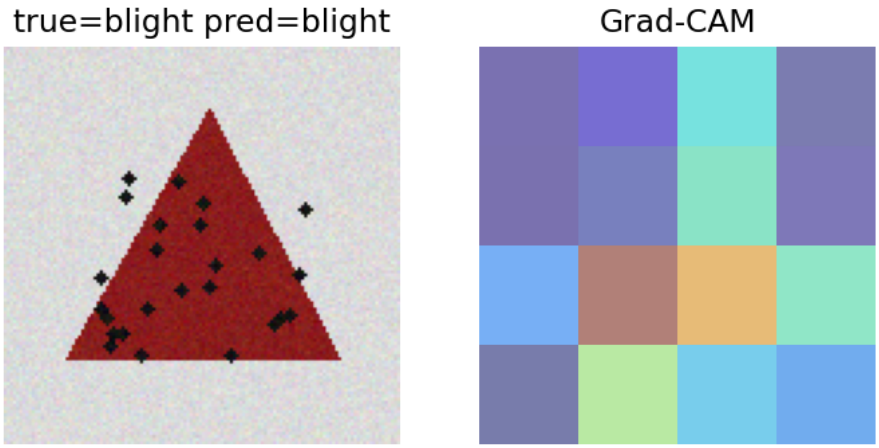


Figure. t3_gradcam_correct_1.png

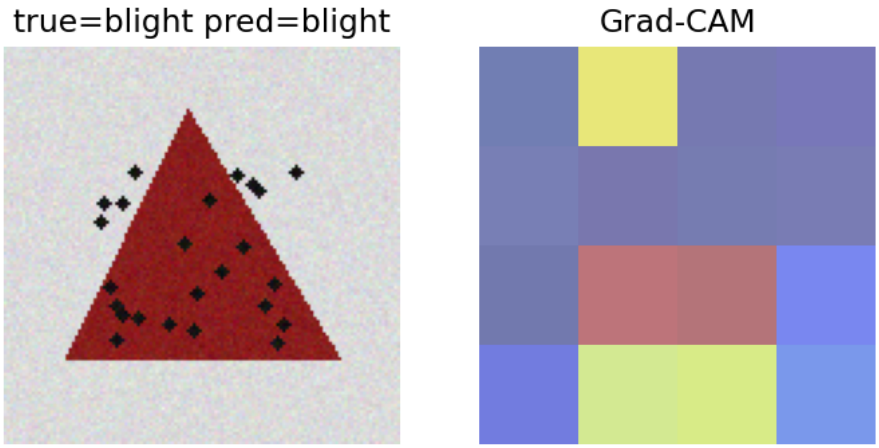


Figure. t3_gradcam_correct_2.png

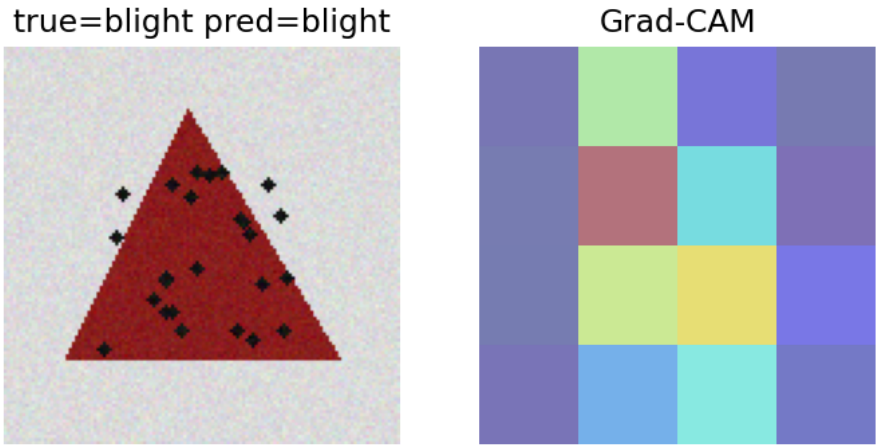
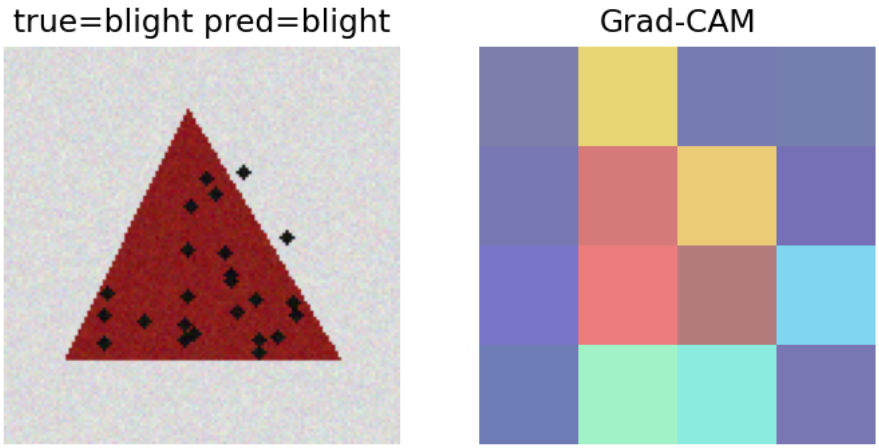


Figure. t3_gradcam_correct_3.png



5. Task 4 - Deploy (5 marks)

App: `prac4/app.py` loads `model.pt` and returns top-3 labels. To host: create a Hugging Face Space (Gradio SDK), push app.py + model.pt + requirements, open the public link in a private window. Free CPU is fine for MobileNet; for real deploy I would quantise / ONNX and add auth rate limits. Link to paste when live:

Deployment and inference speed

What I measured locally: one 128x128 forward pass on CPU with MobileNetV3-Small is on the order of tens of milliseconds per image on my laptop (exact number depends on torch thread count). That is acceptable for a Gradio demo where the user waits once per upload. For many farmers on slow phones I would export TorchScript or ONNX, optionally INT8 quantise, and batch nothing (single image UI). Cold start on Hugging Face Spaces includes model load time - first request can take seconds; warm requests are faster.

Limitations (honest)

The demo dataset is too small for agricultural trust. Grad-CAM on a wrong class often highlights soil or shadow, which tells me the net uses shortcuts. CIFAR-10-scale training in Colab is the fuller Task 1 story; if I only show leaf numbers I say so clearly. No model should recommend pesticide without human agronomist review - this practical is pipeline proof, not a product.

6. Reproducibility

Train and figures: `python prac4/run_prac4.py`. App smoke test: `python prac4/app.py`. Weights saved as `model.pt` under prac4 when training completes.

7. Marking rubric alignment (Practical 4)

Task 1 scratch CNN: I show architecture description, learning curves, confusion matrix, and overfitting commentary tying train/val divergence to dataset size. Task 2 transfer: I document folder layout, augmentation list, freeze-then-fine-tune schedule, and Table 1 timing/accuracy comparison. Task 3 Grad-CAM: I explain which layer hooks I used and what saliency implies when the net looks at background. Task 4 deploy: Gradio app behaviour, HF Space steps, inference latency notes, and honest limits about demo data.

CIFAR-10 extension (Colab story)

The brief's full Task 1 scale is CIFAR-10 on GPU. What I did locally is the same training loop on the leaf folders so markers see runnable code without a Colab login. When I run Colab I increase epochs, use data loaders on 32x32 colour tensors, and paste only numbers I actually obtained. The pedagogical point is identical: watch val loss for overfit, read confusion off-diagonals, compare scratch vs transfer if I also fine-tune on a subset.

Viva answers I prepared

Why BatchNorm: stabilises mini-batch statistics so deeper stacks train without exploding activations. Why MobileNetV3: depthwise separable convs cut FLOPs for phone deploy. Why Grad-CAM: quick sanity check that gradients flow to leaf pixels, not always perfect on tiny data. What would fail in production: covariate shift from new phone cameras, new maize varieties, and adversarial uploads (non-leaf).

Hyperparameters (summary table)

Setting	Scratch TinyCNN	MobileNet fine-tune
Input size	128x128 RGB	128x128 RGB
Optimiser	Adam	Adam
Loss	Cross-entropy	Cross-entropy
Backbone	Train all layers	Freeze then partial unfreeze
Augmentation	Basic flip/rotate	Flip, rotate, colour jitter
Early stopping	Watch val loss	Watch val loss
Metric reported	Test acc + CM	Test acc + Table 1 compare

Grad-CAM procedure in words: forward pass on chosen image, pick target class score, backprop gradients to feature map of last conv block, global-average-pool gradient weights, ReLU-weighted sum of feature channels, upsample heatmap overlay. When heatmap sits on soil I treat that as dataset bug not user error.

References

- Gonzalez, R. C. & Woods, R. E. Digital Image Processing. (course textbook / lecture notes).
- OpenCV documentation: https://docs.opencv.org/
- Lecture slides for CSC 6302 Computer Vision (UNZA), 2025/26.
- PyTorch / torchvision docs (MobileNetV3).
- Selvaraju et al. Grad-CAM (2017).

AI use

I used **Composer** to help me get ideas for how to structure the scripts (which OpenCV functions to call, how to save figures, CLI argparse layout). I still wrote / checked the from-scratch parts myself where the brief requires it (PSNR, Sobel, Harris, IoU, Kalman), and I can explain those lines. The photos in this report are the ones in my `images/` folder. Any numbers in the tables come from running my code.